

# AEEM E417 - Notes

## SQL For Database Management

aristotel.mk

January 18, 2026

### Abstract

Practice is done using Oracle Example tables called Human Resources (HR) at [freessql.com](https://freessql.com). If the SQL table is not available in SQL format every table name should have addition of HR.table\_name.

## Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>SELECT statement</b>                      | <b>2</b>  |
| <b>2</b> | <b>WHERE, OR, AND, ORDER BY statements</b>   | <b>6</b>  |
| <b>3</b> | <b>Functions in SQL</b>                      | <b>10</b> |
| <b>4</b> | <b>Dates and char manipulation</b>           | <b>13</b> |
| <b>5</b> | <b>Join statements</b>                       | <b>17</b> |
| <b>6</b> | <b>GROUP BY, HAVING, ANY, ALL statements</b> | <b>18</b> |
| <b>7</b> | <b>UNION, INTERSECT, MINUS statements</b>    | <b>20</b> |

# 1 SELECT statement

The SELECT statement is a fundamental SQL command used to query and retrieve data from one or more tables within a database. This versatile command allows users to specify exactly which data they wish to see, whether it be all fields from a table or a specific subset of columns. By defining various elements within the statement, such as selecting distinct values or renaming columns with aliases, users can tailor their results to suit their analytical needs.

The syntax of the statement:

```
SELECT * |  
        {[DISTINCT] column |  
        expression [alias],...}  
FROM table;
```

Where each element means:

SELECT - list of one or more columns

\* - all columns

DISTINCT - suppresses duplicates

column | expression - adds to the list a specific column or those which follow the statement request

alias - adds to the list with the alias heading name

FROM table - specifies from which table the list is created.

Example:

```
SELECT *  
FROM departments;
```

Running this SQL statement it will result in the following output:

| NUM | DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|-----|---------------|-----------------|------------|-------------|
| 1   | 10            | Administration  | 200        | 1700        |
| 2   | 20            | Marketing       | 201        | 1800        |
| 3   | 50            | Shipping        | 124        | 1500        |
| 4   | 60            | IT              | 103        | 1400        |
| 5   | 80            | Sales           | 149        | 2500        |
| 6   | 90            | Executive       | 100        | 1700        |
| 7   | 110           | Accounting      | 205        | 1700        |
| 8   | 190           | Contracting     | NULL       | 1700        |

Another example:

```
SELECT department_id, location_id  
FROM departments;
```

Running this SQL statement it will result in the following output:

| NUM | DEPARTMENT_ID | LOCATION_ID |
|-----|---------------|-------------|
| 1   | 10            | 1700        |
| 2   | 20            | 1800        |
| 3   | 50            | 1500        |
| 4   | 60            | 1400        |
| 5   | 80            | 2500        |
| 6   | 90            | 1700        |
| 7   | 110           | 1700        |
| 8   | 190           | 1700        |

Arithmetic expressions in SQL

| Operator | Description |
|----------|-------------|
| +        | Add         |
| -        | Subtract    |
| *        | Multiply    |
| /        | Divide      |

Example:

```
SELECT last_name, salary, salary + 300
FROM employees;
```

Resulting in the following list:

| NUM | last_name | salary   | salary + 300 |
|-----|-----------|----------|--------------|
| 1   | King      | 24000.00 | 24300.00     |
| 2   | Kochhar   | 17000.00 | 17300.00     |
| 3   | De Haan   | 17000.00 | 17300.00     |
| 4   | Hunold    | 9000.00  | 9300.00      |
| ⋮   | ⋮         | ⋮        | ⋮            |
| 19  | Higgins   | 12000.00 | 12300.00     |
| 20  | Gietz     | 8300.00  | 8600.00      |

Another example:

```
SELECT last_name, 12*salary*commission_pct
FROM employees;
```

Resulting in the following list:

| num | last_name | 12 * salary * commission_pct |
|-----|-----------|------------------------------|
| 1   | King      | NULL                         |
| 2   | Kochhar   | NULL                         |
| ⋮   | ⋮         | ⋮                            |
| 14  | Zlotkey   | 25200.0000                   |
| 15  | Abel      | 39600.0000                   |
| 16  | Taylor    | 20640.0000                   |
| 17  | Grant     | 12600.0000                   |
| ⋮   | ⋮         | ⋮                            |
| 19  | Higgins   | NULL                         |
| 20  | Gietz     | NULL                         |

The usage of AS sets the column names in a changeable custom name:

```
SELECT last_name AS "Name",
       commission_pct "Annual_Salary"
FROM employees;
```

Example:

| num | Name    | Annual Salary |
|-----|---------|---------------|
| 1   | King    | NULL          |
| 2   | Kochhar | NULL          |
| ⋮   | ⋮       | ⋮             |
| 14  | Zlotkey | 0.20          |
| 15  | Abel    | 0.30          |
| 16  | Taylor  | 0.20          |
| 17  | Grant   | 0.15          |
| ⋮   | ⋮       | ⋮             |
| 19  | Higgins | NULL          |
| 20  | Gietz   | NULL          |

Concatenation - merging text is done in the following two ways:

```
SELECT last_name || job_id
AS "Employees"
FROM employees;

SELECT CONCAT(last_name, job_id)
AS Employees
FROM employees;
```

Usage of || permits more than one text concatenation, CONCAT() requires nesting of multiple CONCAT statements. Also || is not available to be used in MariaDB, so CONCAT() is the only viable way of doing concatenation.

Example:

| num | Employees       |
|-----|-----------------|
| 1   | KingAD_PRES     |
| 2   | KochharAD_VP    |
| 3   | De HaanAD_VP    |
| 4   | HunoldIT_PROG   |
| 5   | ErnstIT_PROG    |
| ⋮   | ⋮               |
| 20  | GietzAC_ACCOUNT |

Operator DISTINCT - filters out only unique inputs

```
SELECT DISTINCT department_id, job_id
FROM employees;
```

Example output:

| num | department_id | job_id     |
|-----|---------------|------------|
| 1   | 90            | AD_PRES    |
| 2   | 90            | AD_VP      |
| 3   | 60            | IT_PROG    |
| 4   | 50            | ST_MAN     |
| 5   | 50            | ST_CLERK   |
| 6   | 80            | SA_MAN     |
| 7   | 80            | SA_REP     |
| 8   | NULL          | SA_REP     |
| 9   | 10            | AD_ASST    |
| 10  | 20            | MK_MAN     |
| 11  | 20            | MK_REP     |
| 12  | 110           | AC_MGR     |
| 13  | 110           | AC_ACCOUNT |

Note there are 20 employees in total.

In order to get information about one table it's structure, data type & other specificity the DESCRIBE or DESC statement is used.

```
DESC[RIBE] tablename
```

Example:

```
DESC employees;
```

Example output:

| num | Field          | Type         | Null | Key  | Default | Extra |
|-----|----------------|--------------|------|------|---------|-------|
| 1   | EMPLOYEE_ID    | int(11)      | NO   | NULL | NULL    | NULL  |
| 2   | FIRST_NAME     | varchar(20)  | YES  | NULL | NULL    | NULL  |
| 3   | LAST_NAME      | varchar(255) | NO   | NULL | NULL    | NULL  |
| 4   | EMAIL          | varchar(255) | NO   | NULL | NULL    | NULL  |
| 5   | PHONE_NUMBER   | varchar(20)  | YES  | NULL | NULL    | NULL  |
| 6   | HIRE_DATE      | date         | NO   | NULL | NULL    | NULL  |
| 7   | JOB_ID         | varchar(10)  | NO   | NULL | NULL    | NULL  |
| 8   | SALARY         | decimal(8,2) | YES  | NULL | NULL    | NULL  |
| 9   | COMMISSION_PCT | decimal(2,2) | YES  | NULL | NULL    | NULL  |
| 10  | MANAGER_ID     | int(11)      | YES  | NULL | NULL    | NULL  |
| 11  | DEPARTMENT_ID  | int(11)      | YES  | MUL  | NULL    | NULL  |

## 2 WHERE, OR, AND, ORDER BY statements

Using WHERE and the other mentioned logical operators there is the capability to form logical statements which are a close equivalent to if statements in most common programming languages.

Simple example:

```
SELECT last_name, job_id, department_id
FROM employees
WHERE job_id = 'SA_REP';
```

Example output:

| num | last_name | job_id | department_id |
|-----|-----------|--------|---------------|
| 1   | Abel      | SA_REP | 80            |
| 2   | Taylor    | SA_REP | 80            |
| 3   | Grant     | SA_REP | NULL          |

Instead of just matching the condition with = , there is the possibility of using comparison operators:

| Operator | Meaning             |
|----------|---------------------|
| =        | Equal to            |
| <>       | Not equal to        |
| >        | Greater than        |
| >=       | Greater or equal to |
| <        | Less than           |
| <=       | Less or equal to    |

Example:

```
SELECT last_name, salary
FROM employees
WHERE salary <= 3000;
```

Example output:

| num | last_name | salary  |
|-----|-----------|---------|
| 1   | Matos     | 2600.00 |
| 2   | Vargas    | 2500.00 |

## Other comparison Operators

| Operator            | Meaning                     |
|---------------------|-----------------------------|
| BETWEEN ... AND ... | Between two values          |
| IN (set)            | Match any of list of values |
| LIKE                | Match a character pattern   |
| IS NULL             | Is a null value             |

Numerical BETWEEN examples:

```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 2500 AND 3500;
```

Example output:

| num | last_name | salary  |
|-----|-----------|---------|
| 1   | Rajs      | 3500.00 |
| 2   | Davies    | 3100.00 |
| 3   | Matos     | 2600.00 |
| 4   | Vargas    | 2500.00 |

Character BETWEEN example:

```
SELECT last_name
FROM employees
WHERE last_name BETWEEN 'King' AND 'Smith';
```

Example output:

| num | last_name | salary   |
|-----|-----------|----------|
| 1   | King      | 24000.00 |
| 2   | Kochhar   | 17000.00 |
| 3   | Lorentz   | 4200.00  |
| 4   | Mourgos   | 5800.00  |
| 5   | Rajs      | 3500.00  |
| 6   | Matos     | 2600.00  |

Numerical IN example:

```
SELECT employee_id, last_name, salary, manager_id
FROM employees
WHERE manager_id IN (100, 101, 201);
```

Example output:

| num | employee_id | last_name | salary   | manager_id |
|-----|-------------|-----------|----------|------------|
| 1   | 101         | Kochhar   | 17000.00 | 100        |
| 2   | 102         | De Haan   | 17000.00 | 100        |
| 3   | 124         | Mourgos   | 5800.00  | 100        |
| 4   | 149         | Zlotkey   | 10500.00 | 100        |
| 5   | 200         | Whalen    | 4400.00  | 101        |
| 6   | 201         | Harstein  | 13000.00 | 100        |
| 7   | 202         | Fay       | 6000.00  | 200        |
| 8   | 205         | Higgins   | 12000.00 | 101        |

Character IN example:

```
SELECT employee_id, manager_id, department_id
FROM employees
WHERE last_name IN ('Hartstein', 'Vargas');
```

Example output:

| num | employee_id | manager_id | department_id |
|-----|-------------|------------|---------------|
| 1   | 144         | 124        | 50            |
| 2   | 201         | 100        | 20            |

Using \_\_ operator with LIKE example:

```
SELECT first_name
FROM employees
WHERE last_name LIKE 'E_____';
```

Example output:

| num | first_name |
|-----|------------|
| 1   | Eleni      |
| 2   | Ellen      |

Example:

Using % operator with LIKE example:

```
SELECT last_name
FROM employees
WHERE last_name LIKE '%t';
```

Example output:

| num | last_name |
|-----|-----------|
| 1   | Ernst     |
| 2   | Grant     |

Example of IS NULL statement:

```
SELECT last_name, manager_id
FROM employees
WHERE manager_id IS NULL;
```

Example output:

| num | last_name | manager_id |
|-----|-----------|------------|
| 1   | King      | NULL       |

Logical Operators

| Operator | Meaning                           |
|----------|-----------------------------------|
| AND      | Mathematical equivalent of $\cap$ |
| OR       | Mathematical equivalent of $\cup$ |
| NOT      | Mathematical equivalent of $'$    |



AND Logic:

| $\cap$ | TRUE  | FALSE | NULL  |
|--------|-------|-------|-------|
| TRUE   | TRUE  | FALSE | NULL  |
| FALSE  | FALSE | FALSE | FALSE |
| NULL   | NULL  | NULL  | NULL  |

OR Logic:

| $\cup$ | TRUE | FALSE | NULL |
|--------|------|-------|------|
| TRUE   | TRUE | TRUE  | TRUE |
| FALSE  | TRUE | FALSE | NULL |
| NULL   | TRUE | NULL  | NULL |

NOT logic:

| $\neg$ | RESULT |
|--------|--------|
| TRUE   | FALSE  |
| FALSE  | TRUE   |
| NULL   | NULL   |

Nested OR, AND example:

```
SELECT last_name, job_id, salary
FROM employees
WHERE (job_id = 'SA_REP'
OR job_id = 'AD_PRES')
AND salary > 10000;
```

Example output:

| NUM | last_name | job_id  | salary   |
|-----|-----------|---------|----------|
| 1   | King      | AD_PRES | 24000.00 |
| 2   | Abel      | SA_REP  | 11000.00 |

Priority of SQL statements

| Order | Operator Type                 | Explanation                                                                                         |
|-------|-------------------------------|-----------------------------------------------------------------------------------------------------|
| 1     | Arithmetic operators          | Operations like addition (+), subtraction (-), multiplication ( $\times$ ), and division ( $\div$ ) |
| 2     | Concatenation operator        | Combines strings (e.g., '  ' in SQL)                                                                |
| 3     | Comparison conditions         | Includes operators like '=', '>', '<', '>=', '<=', which compare values                             |
| 4     | IS [NOT] NULL, LIKE, [NOT] IN | Checks for null values, pattern matching, or membership in a set                                    |
| 5     | [NOT] BETWEEN                 | Identifies if a value falls within a specific range or not                                          |
| 6     | Not equal to                  | Represented as '<>', checks if two values are not equal                                             |
| 7     | NOT logical condition         | Negates a condition, changing TRUE to FALSE and vice versa                                          |
| 8     | AND logical condition         | Requires both conditions to be TRUE for the result to be TRUE                                       |
| 9     | OR logical condition          | Requires at least one of the conditions to be TRUE for the result to be TRUE                        |

Syntax of the ORDER BY ordering statement:

| Term          | Description                                                                |
|---------------|----------------------------------------------------------------------------|
| ORDER BY      | Specifies the order in which the retrieved rows are displayed              |
| ASC           | Orders the rows in ascending order (this is the default order)             |
| DESC          | Orders the rows in descending order                                        |
| COLUMN NUMBER | Using the number of the column you could order without mentioning the name |

Example with COLUMN NUMBER:

```
SELECT last_name, job_id, department_id, hire_date
FROM EMPLOYEES
ORDER BY 3;
```

Example output:

| NUM | last_name | job_id     | department_id | hire_date  |
|-----|-----------|------------|---------------|------------|
| 1   | Grant     | SA_REP     | NULL          | 1999-05-24 |
| 2   | Whalen    | AD_ASST    | 10            | 1987-09-17 |
| ⋮   | ⋮         | ⋮          | ⋮             | ⋮          |
| 20  | Gietz     | AC_ACCOUNT | 110           | 1994-06-07 |

Example sorting by multiple columns:

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id, salary DESC;
```

Example output:

| NUM | last_name | department_id | salary   |
|-----|-----------|---------------|----------|
| 1   | Grant     | NULL          | 7000.00  |
| 2   | Whalen    | 10            | 4400.00  |
| ⋮   | ⋮         | ⋮             | ⋮        |
| 19  | Higgins   | 110           | 12000.00 |
| 20  | Gietz     | 110           | 8300.00  |

It sorted them firstly by the department\_id than by salary in a descending order.

### 3 Functions in SQL

#### Character Functions

| Function                                                             | Purpose                                                                                                                  |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| LOWER (column   expression)                                          | Converts to lowercase.                                                                                                   |
| UPPER (column   expression)                                          | Converts to uppercase.                                                                                                   |
| INITCAP (column   expression)                                        | Converts first letter of each word into uppercase.                                                                       |
| SUBSTR(column   expression, [m,n])                                   | Extracts characters starting from m-th and limiting to n-number of characters. Doing -m starts from the end of the word. |
| LENGTH (column   expression)                                         | Returns the number of characters in the expression.                                                                      |
| INSTR(column   expression, string, [m,n])                            | Numeric position of a provided text from m-th position (default is 1) to the n-th occurrence.                            |
| LPAD(column   expression, n, 'string')                               | Left justified addition of a custom character to the prompted characters.                                                |
| RPAD(column   expression, n, 'string')                               | Right justified addition of a custom character to the prompted characters.                                               |
| TRIM(leading   trailing   both, 'trim_character' FROM 'trim_source') | Enables you to trim heading or trailing character (or both) from a character string.                                     |
| REPLACE (text, 'search_string', 'replacement_string')                | Searches a text expression for a character string and, if found, replaces it with a specified replacement string.        |

LOWER, UPPER, INITCAP examples:

```
SELECT LOWER('SQL_Course')
FROM DUAL;
```

```

SELECT UPPER('SQL_Course')
FROM DUAL;

SELECT INITCAP('SQL_Course')
FROM DUAL;

```

Example output:

| NUM | LOWER('SQL Course') | UPPER('SQL Course') | INITCAP('SQL Course') |
|-----|---------------------|---------------------|-----------------------|
| 1   | sql course          | SQL COURSE          | Sql Course            |

SUBSTR, LENGTH, INSTR examples:

```

SELECT SUBSTR('HelloWorld',1,5)
FROM DUAL;

SELECT LENGTH('HelloWorld')
FROM DUAL;

SELECT INSTR('HelloWorld','W')
FROM DUAL;

```

Examples output:

| NUM | SUBSTR('HelloWorld',1,5) | LENGTH('HelloWorld') | INSTR('HelloWorld','W') |
|-----|--------------------------|----------------------|-------------------------|
| 1   | Hello                    | 10                   | 6                       |

LPAD, RPAD examples:

```

SELECT LPAD(24000,10,'*')
FROM DUAL;

SELECT RPAD(24000,10,'*')
FROM DUAL;

```

Examples output:

| NUM | LPAD(24000,10,'*') | RPAD(24000,10,'*') |
|-----|--------------------|--------------------|
| 1   | *****24000         | 24000*****         |

REPLACE, TRIM examples:

```

SELECT REPLACE(' JACK_and_JUE','J','BL')
FROM DUAL;

SELECT TRIM('H' FROM 'HelloWorld')
FROM DUAL;

```

Examples output:

| NUM | REPLACE('JACK and JUE','J','BL') | TRIM('H' FROM 'HelloWorld') |
|-----|----------------------------------|-----------------------------|
| 1   | BLACK and BLUE                   | elloWorld                   |

## Number Functions

| Function                                | Purpose                                                                                                   |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ROUND(column   expression, n)           | Rounds the specified column / expression to a value of n number of decimal places, n defaults to zero.    |
| TRUNC(column   expression, n)           | Truncates the specified column / expression to a value of n number of decimal places, n defaults to zero. |
| MOD(m,n)                                | Returns the modulus (remainder of m divided by n).                                                        |
| COUNT([*   DISTINCT   ALL], expression) | Number of rows, where expression evaluates to something other than NULL.                                  |
| SUM([DISTINCT   ALL], n)                | Sums values of n, ignoring NULL values.                                                                   |
| AVG([DISTINCT   ALL], n)                | Averages the numerical values of n, ignoring NULL values.                                                 |
| MAX([DISTINCT   ALL], expression)       | Maximum value of expression, ignoring NULL values.                                                        |
| MIN([DISTINCT   ALL], expression)       | Minimum value of expression, ignoring NULL values.                                                        |
| VARIANCE([DISTINCT   ALL], n)           | Variance of n, ignoring NULL values.                                                                      |
| STDDEV([DISTINCT   ALL], n)             | Standard deviation of n, ignoring NULL values.                                                            |

ROUND, TRUNC examples:

```
SELECT ROUND(45.926, 2)
FROM DUAL;

SELECT TRUNC(45.926, 2)
FROM DUAL;
```

Examples output:

| NUM | ROUND(45.926,2) | TRUNC(45.926,2) |
|-----|-----------------|-----------------|
| 1   | 45.93           | 45.92           |

MOD, COUNT examples:

```
SELECT last_name, salary, MOD(salary, 5000)
FROM employees
WHERE job_id = 'SA_REP';

SELECT COUNT(DISTINCT department_id)
FROM employees;
```

Examples output:

| NUM | last_name | salary   | MOD(salary,5000) |
|-----|-----------|----------|------------------|
| 1   | Abel      | 11000.00 | 1000.00          |
| 2   | Taylor    | 8600.00  | 3600.00          |
| 3   | Grant     | 7000.00  | 2000.00          |

| NUM | COUNT(DISTINCT department_id) |
|-----|-------------------------------|
| 1   | 7                             |

SUM, AVG, MAX, MIN example:

```
SELECT AVG(salary), MAX(salary),
MIN(salary), SUM(salary)
```

```
FROM employees
WHERE job_id LIKE '%REP%';
```

Example output:

| NUM | AVG(salary) | MAX(salary) | MIN(salary) | SUM(salary) |
|-----|-------------|-------------|-------------|-------------|
| 1   | 8150.00     | 11000.00    | 6000.00     | 32600.00    |

## 4 Dates and char manipulation

Date in SQL is stored in the following format:

| CENTURY | YEAR | MONTH | DAY | HOURL | MINUTE | SECOND | MICROSECONDS |
|---------|------|-------|-----|-------|--------|--------|--------------|
| 20      | 26   | 01    | 19  | 15    | 20     | 20     | 000          |

```
SELECT SYSDATE FROM DUAL;

or in MariaDB

SELECT SYSDATE();
```

Output:

| NUM | SYSDATE / ()            |
|-----|-------------------------|
| 1   | 2026-01-19 15:20:20:000 |

Date Functions:

| Function                                   | Purpose                                                                                                               |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| SYSDATE                                    | Shows current system (machine) time.                                                                                  |
| MONTHS_BETWEEN (date1, date2)              | Counts and displays the number of months between two dates.                                                           |
| ADD_MONTHS (date, n)                       | Adds a specified number of months to the provided date.                                                               |
| NEXT_DAY                                   | Finds the date of the next specified day of the week in ('char') following the date.                                  |
| LAST_DAY (date)                            | Finds the date of the last day of the month that contains the date.                                                   |
| TO_CHAR(number   date, [fmt.], [nlsparms]) | Converts a number or date value to a VARCHAR(2) character string with format model (fmt).                             |
| TO_NUMBER(char, [fmt], [nlsparms])         | Converts a character string containing digits to a number in the format specified by the optional format model (fmt). |
| TO_DATE(char, [fmt], [nlsparms])           | Converts a character string representing a date to a date value according to the format specified (fmt).              |

MONTHS\_BETWEEN, ADD\_MONTHS, NEXT\_DAY, LAST\_DAY examples:

```
SELECT MONTHS_BETWEEN (TO_DATE('01-SEP-95', 'dd-MON-yy'), TO_DATE('
11-JAN-94', 'dd-MON-yy'))
FROM DUAL;

in MariaDB
SELECT TIMEDIFF(MONTH, STR_TO_DATE('11-JAN-94', '%d-%b-%y'
), STR_TO_DATE('01-SEP-95', '%d-%b-%y'));

SELECT ADD_MONTHS (TO_DATE('11-JAN-94', 'dd-MON-yy'), 6)
```

```

FROM DUAL;

in MariaDB
SELECT DATE_ADD(STR_TO_DATE('11-JAN-94', '%d-%b-%y'), INTERVAL 6
MONTH) AS new_date;

SELECT NEXT_DAY (TO_DATE('01-SEP-95', 'dd-MON-yy'), 'FRIDAY')
FROM DUAL;

in MariaDB
SELECT DATE_ADD(STR_TO_DATE('01-SEP-95', '%d-%b-%y'),
INTERVAL (7 - WEEKDAY(STR_TO_DATE('01-SEP-95', '%d-%b-%y')) + 4) %
7) DAY AS next_friday;

SELECT LAST_DAY(TO_DATE('01-FEB-95', 'dd-MON-yy'))
FROM DUAL;

in MariaDB
SELECT LAST_DAY(STR_TO_DATE('01-FEB-95', '%d-%b-%y')) AS LAST_DAY;

```

Examples output:

| NUM | MONTHS_BETWEEN | ADD_MONTHS             | NEXT_DAY              | LAST_DAY               |
|-----|----------------|------------------------|-----------------------|------------------------|
| 1   | 19.677         | 7/11/2094, 12:00:00 AM | 9/2/2095, 12:00:00 AM | 2/28/2095, 12:00:00 AM |

TO\_CHAR, TO\_NUMBER, TO\_DATE examples:

```

SELECT last_name,
TO_CHAR(hire_date, 'fmDD_Month_YYYY') HIREDATE
FROM EMPLOYEES;

SELECT last_name, hire_date
FROM employees
WHERE hire_date = TO_DATE('May_24_1999', 'fxMonth_DD_YYYY');

SELECT last_name, TO_NUMBER(TO_CHAR(hire_date, 'YYYY')) AS
hire_year
FROM EMPLOYEES
WHERE hire_date = TO_DATE('May_24_1999', 'fxMonth_DD_YYYY');

in MariaDB it is STR_TO_DATE('May_24_1999', '%M_%d_%Y');

```

Examples Output:

| NUM | last_name | HIREDATE     |
|-----|-----------|--------------|
| 1   | King      | 17 June 1987 |
| ⋮   | ⋮         | ⋮            |
| 20  | Gietz     | 7 June 1994  |

| NUM | last_name | hire_date  |
|-----|-----------|------------|
| 1   | Grant     | 1999-05-24 |

| NUM | last_name | hire_date |
|-----|-----------|-----------|
| 1   | Grant     | 1999      |

Date Formats:

| Format | Purpose                                          |
|--------|--------------------------------------------------|
| YYYY   | Full year in numbers                             |
| YEAR   | Year spelled out                                 |
| MM     | Two-digit value for month                        |
| MONTH  | Full name of the month                           |
| MON    | Three-letter abbreviation of the month           |
| DY     | Three-letter abbreviation of the day of the week |
| DAY    | Full name of the day of the week                 |
| DD     | Numeric day of the month                         |

General NULL / IF Functions:

| Function                           | Purpose                                                                                                    |
|------------------------------------|------------------------------------------------------------------------------------------------------------|
| NVL (column,value to replace NULL) | Converts a null value to an actual value                                                                   |
| NVL2 (expr1, expr2, expr3)         | If expr1 is not null, returns expr 2, If expr1 is null, returns expr 3                                     |
| NULLIF (expr1,expr2)               | Compares two expressions and returns null if they are equal, or the first expression if they are not equal |
| COALESCE ()                        | Returns the first non-null expression in the expression list                                               |
| CASE                               | Equivalent to case statements or IF-THEN-ELSE                                                              |
| DECODE                             | LOOKUP equivalent                                                                                          |

NVL, NVL2 examples

```
SELECT last_name, salary, NVL(commission_pct, 0),
(salary*12) + (salary*12*NVL(commission_pct,0)) AN_SAL
FROM employees;

SELECT last_name, salary, commission_pct,
NVL2(commission_pct, 'SAL+COM', 'SAL') income
FROM employees WHERE department_id IN (50,80);
```

Examples output:

| NUM | last_name | salary   | NVL(commision_pct, 0) | AN_SAL    |
|-----|-----------|----------|-----------------------|-----------|
| 1   | King      | 24000.00 | 0                     | 288000.00 |
| ⋮   | ⋮         | ⋮        | ⋮                     | ⋮         |
| 15  | Grant     | 7000.00  | 0.15                  | 96600.00  |
| ⋮   | ⋮         | ⋮        | ⋮                     | ⋮         |
| 20  | Gietz     | 8300.00  | 0                     | 99600.00  |

| NUM | last_name | salary  | NVL(commision_pct, 0) | income  |
|-----|-----------|---------|-----------------------|---------|
| 1   | Mourgos   | 5800.00 | NULL                  | SAL     |
| ⋮   | ⋮         | ⋮       | ⋮                     | ⋮       |
| 8   | Taylor    | 8600.00 | 0.2                   | SAL+COM |

NULLIF, COALASCE examples

```
SELECT first_name, LENGTH(first_name) "expr1",
last_name, LENGTH(last_name) "expr2",
NULLIF(LENGTH(first_name), LENGTH(last_name)) result
FROM employees;

SELECT last_name, salary, commission_pct,
COALESCE((salary+(commission_pct*salary)), salary+2000, salary) "
New_Salary"
FROM employees;
```

Examples output:

| NUM | first_name | expr1 | last_name | expr2 | result |
|-----|------------|-------|-----------|-------|--------|
| 1   | Steven     | 6     | King      | 4     | 6      |
| ⋮   | ⋮          | ⋮     | ⋮         | ⋮     | ⋮      |
| 9   | Curtis     | 6     | Davies    | 6     | NULL   |
| ⋮   | ⋮          | ⋮     | ⋮         | ⋮     | ⋮      |
| 20  | William    | 7     | Gietz     | 5     | 7      |

| NUM | last_name | salary   | commision_pct | New Sallary |
|-----|-----------|----------|---------------|-------------|
| 1   | King      | 24000.00 | NULL          | 26000.00    |
| ⋮   | ⋮         | ⋮        | ⋮             | ⋮           |
| 12  | Zlotkey   | 10500.00 | 0.2           | 12600.00    |
| ⋮   | ⋮         | ⋮        | ⋮             | ⋮           |
| 20  | Gietz     | 8300.00  | NULL          | 10300.00    |

CASE, DECODE examples

```
SELECT last_name, job_id, salary,
CASE job_id
WHEN 'IT_PROG' THEN 1.10*salary
WHEN 'ST_CLERK' THEN 1.15*salary
WHEN 'SA_REP' THEN 1.20*salary
ELSE salary
END "REVISED_SALARY"
FROM employees;

SELECT last_name, job_id, salary,
DECODE( job_id, 'IT_PROG', 1.10*salary,
' ST_CLERK', 1.15*salary,
' SA_REP', 1.20*salary,
```



```

        salary)
"REVISED_SALARY"
FROM employees;

```

Examples output - same output for both examples:

| NUM | last_name | job_id     | salary   | REVISED_SALARY |
|-----|-----------|------------|----------|----------------|
| 1   | King      | AD_PRES    | 24000.00 | 24000.00       |
| ⋮   | ⋮         | ⋮          | ⋮        | ⋮              |
| 13  | Abel      | SA_REP     | 10500.00 | 13200.00       |
| ⋮   | ⋮         | ⋮          | ⋮        | ⋮              |
| 20  | Gietz     | AC_ACCOUNT | 8300.00  | 8300.00        |

## 5 Join statements

An Equi Join is a specific type of join that uses equality conditions to match records from two or more tables based on one or more columns. Typically, it employs the = operator in the join condition. The result of an equi join includes only the rows where the specified columns have the same values across the participating tables. This is useful when you want to combine related data, such as when matching customer information from one table with their corresponding orders in another. For example, if you have a Customers table and an Orders table, you can perform an equi join on the CustomerID field to retrieve a combined view of customer details along with their orders.

```

SELECT e.employee_id, e.last_name, e.department_id,
       d.department_id, d.location_id
FROM   employees e, departments d
WHERE  e.department_id = d.department_id;

```

A Self Join is a unique type of join where a table is joined with itself. This might seem a bit counter-intuitive at first but is incredibly useful for hierarchical data or when comparing rows within the same table. To implement a self join, you'll assign different aliases to the same table in the SQL query. For instance, if you have an Employees table that has a ManagerID column indicating the manager of each employee, a self join can be used to retrieve a list of employees along with their respective managers' names. The self join makes it possible to query and view relationships within a single dataset, which can be particularly valuable in scenarios like organizational structures or parent-child relationships in data.

```

SELECT worker.last_name || '␣works␣for␣'
       || manager.last_name
FROM   employees worker, employees manager
WHERE  worker.manager_id = manager.employee_id;

```

| Function                | Purpose                                                                                                                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NATURAL JOIN            | Combines rows from two or more tables based on the columns with the same names and data types, eliminating duplicate columns in the result.                                                           |
| JOIN ... USING          | Merges two tables based on a common column(s) specified in the USING clause, resulting in a simplified result set that only includes the matched columns once.                                        |
| JOIN ... ON             | Joins two tables based on a specified condition in the ON clause, allowing for more complex conditions beyond just equality of specific columns.                                                      |
| LEFT OUTER JOIN ... ON  | Retrieves all rows from the left table and the matched rows from the right table; if no match is found, NULL values are returned for columns from the right table.                                    |
| RIGHT OUTER JOIN ... ON | Retrieves all rows from the right table and the matched rows from the left table; if no match is found, NULL values are returned for columns from the left table.                                     |
| FULL OUTER JOIN ... ON  | Combines the results of LEFT and RIGHT OUTER JOINS, returning all rows from both tables with NULLs in place where there are no matches.                                                               |
| CROSS JOIN ... ON       | Produces the Cartesian product of two tables, pairing every row from the first table with every row from the second; the ON clause is typically not applicable here as all combinations are returned. |

## 6 GROUP BY, HAVING, ANY, ALL statements

| Function | Purpose                                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------------------------------|
| GROUP BY | Used to group rows that have the same values in specified columns into summary rows.                                    |
| HAVING   | Used to filter groups of rows after the aggregation has occurred, allowing you to specify criteria on the grouped data. |

Examples:

```
SELECT job_id, AVG(salary)
FROM EMPLOYEES
GROUP BY job_id;

SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary) > 10000;
```

Example output:

| NUM | job_id     | AVG(salary) |
|-----|------------|-------------|
| 1   | AC_ACCOUNT | 8300.00     |
| ⋮   | ⋮          | ⋮           |
| 12  | ST_MAN     | 5800.00     |

| NUM | department_id | AVG(salary) |
|-----|---------------|-------------|
| 1   | 20            | 9500.00     |
| 2   | 80            | 10033.34    |
| 3   | 90            | 19333.34    |
| 4   | 110           | 10150.00    |

Multiple-row comparison Functions:

| Function | Purpose                                                                                                      |
|----------|--------------------------------------------------------------------------------------------------------------|
| IN       | Equal to any member in the list                                                                              |
| ALL      | Must be preceded by a comparison operator, compares a value to each value in a list or returned by a query.  |
| ANY      | Must be preceded by a comparison operator, compares a value to every value in a list or returned by a query. |

```

SELECT last_name, salary, department_id
FROM employees
WHERE salary IN (8300, 8600, 17000);

SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary < ANY (SELECT salary
                     FROM employees
                     WHERE job_id = 'IT_PROG')
AND job_id <> 'IT_PROG';

SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary < ALL (SELECT salary
                     FROM employees
                     WHERE job_id = 'IT_PROG')
AND job_id <> 'IT_PROG'

```

Example output:

| NUM | last_name | salary   | department_id |
|-----|-----------|----------|---------------|
| 1   | Kochhar   | 17000.00 | 90            |
| 2   | De Haan   | 17000.00 | 90            |
| 3   | Taylor    | 8600.00  | 80            |
| 4   | Gietz     | 8300.00  | 110           |

| NUM | employee_id | last_name | job_id     | salary  |
|-----|-------------|-----------|------------|---------|
| 1   | 124         | Murgos    | ST_MAN     | 5800.00 |
| ⋮   | ⋮           | ⋮         | ⋮          | ⋮       |
| 10  | 206         | Gietz     | AC_ACCOUNT | 8300.00 |

| NUM | employee_id | last_name | job_id   | salary  |
|-----|-------------|-----------|----------|---------|
| 1   | 141         | Rajs      | ST_CLERK | 3500.00 |
| ⋮   | ⋮           | ⋮         | ⋮        | ⋮       |
| 4   | 206         | Vargas    | ST_CLERK | 2500.00 |

## 7 UNION, INTERSECT, MINUS statements

Set Operator Functions:

| Function       | Purpose                                                          |
|----------------|------------------------------------------------------------------|
| UNION          | Rows from both queries after eliminating duplications            |
| UNION ALL      | Rows from both queries, including all duplications               |
| INTERSECT      | Rows that are common to both queries                             |
| MINUS (EXCEPT) | Rows in the first query that are not present in the second query |

UNION example:

```
SELECT job_id
FROM employees
UNION
SELECT job_id
FROM job_history;
```

Example output:

| NUM | job_id  |
|-----|---------|
| 1   | AD_PRES |
| ⋮   | ⋮       |
| 12  | ST_MAN  |

UNION ALL example:

```
SELECT job_id, job_id
FROM employees
UNION ALL
SELECT job_id
FROM job_history
ORDER BY job_id;
```

Example output:

| NUM | job_id  |
|-----|---------|
| 1   | AD_PRES |
| ⋮   | ⋮       |
| 20  | ST_MAN  |

INTERSECT example:

```
SELECT job_id
FROM employees
UNION ALL
```

```
SELECT job_id
FROM job_history
ORDER BY job_id;
```

Example output:

| NUM | job_id |
|-----|--------|
|-----|--------|

MINUS example:

```
SELECT employee_id
FROM employees
MINUS
SELECT employee_id
FROM job_history;
```

Example output:

| NUM | job_id  |
|-----|---------|
| 1   | AD_PRES |
| ⋮   | ⋮       |
| 12  | ST_MAN  |